

GalaxEye — Backend Engineer, ML Systems

Design Note & Problem-Solving Answers

GalaxEye Take-Home — Sangeeta Upadhye

PART 1

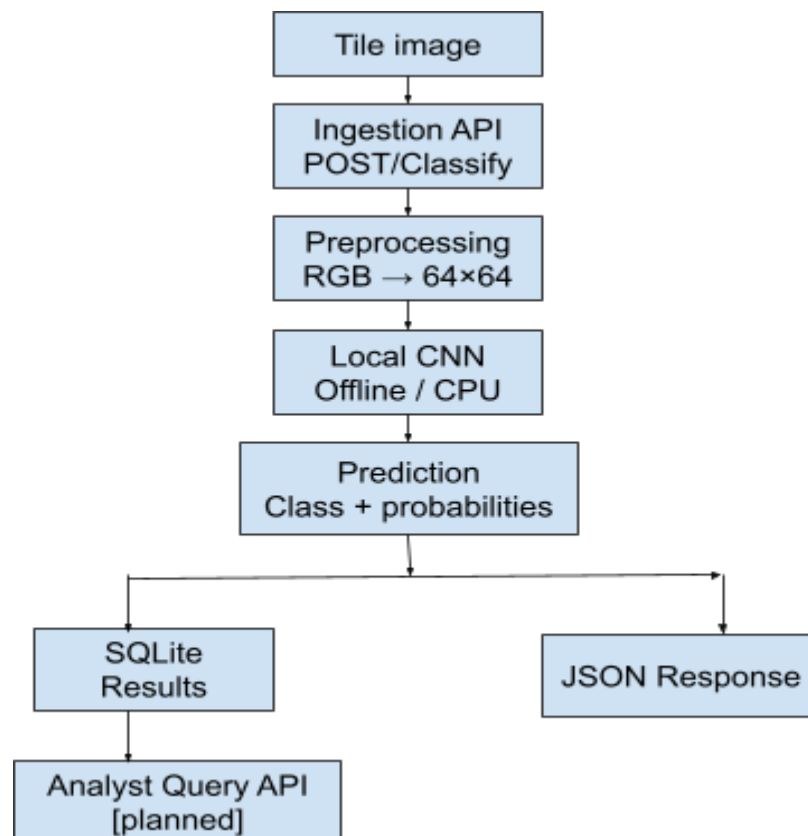
Architecture

Then your short architecture introduction:

I split the system into four logical components:

1. Ingestion API — receives the tile through POST /classify
2. Inference layer — preprocesses the image and runs the local CNN classifier
3. Storage layer — SQLite stores each classification result
4. Query layer — allows an analyst to search historical results
(*designed, not built in the thin working slice*)

Architecture diagram



How a tile flows through the system

- A tile image is uploaded through the POST /classify endpoint.
- The image is converted to RGB, resized to 64×64, and converted to a tensor.
- The CNN gives a probability for each of the seven classes.
- The class with the highest probability is returned as the prediction.
- I keep the full probability list instead of only storing the top class.
- The result is stored in SQLite with filename, predicted class, confidence, probability breakdown, model version, and timestamp.
- The prediction is also returned as JSON.

Why I kept it as a single process

The target environment is isolated hardware, so I preferred a simple setup with fewer moving parts. The API, model inference and SQLite storage can run on the same machine.

If the number of tiles becomes large or different components need separate hardware, these components can be separated later.

Offline design

The model files and class mapping are stored with the application. The model is loaded locally and inference does not call an external API.

I chose SQLite because it works as a local database file and does not require a separate database server.

Handling low-confidence predictions

The highest probability is not always a strong prediction. For example, Forest : 0.91 is different from Forest : 0.38 when another class has 0.34.

For the full system, I would decide a confidence threshold using validation results. Predictions below the threshold could be flagged for review.

For the current implementation, I return and store the complete probability vector. This keeps the uncertainty information available instead of losing it.

What I store

For every prediction I store:

Field	Why
id	Unique result ID
filename	Input tile
predicted_class	Model prediction
confidence	Probability of predicted class
all_class_probabilities	Probabilities for all classes
model_version	Which model produced the result
created_at	When prediction was made

I keep the complete probability vector because it lets an analyst see whether another class was also close to the prediction.

I also store the model version because the model may be retrained or replaced later.

Analyst querying

For the full system, I would add a read-only query endpoint.

An analyst could filter results by:

- predicted class
- confidence
- model version
- date/time
- tile ID or filename

For example, they could look for low-confidence River predictions from a particular model version.

I did not build this in the current working slice because Part 2 focuses on the main path: upload a tile → classify it → store the result → return the prediction.

Assumptions

My main assumptions are:

- The service will run on isolated hardware without internet during inference.
- The seven classes in the supplied dataset are the required classes.
- CPU inference is acceptable for the expected workload.
- SQLite is sufficient for the initial single-machine setup.
- The model may be retrained or replaced, so each result should keep its model version.
- Analysts may need to review uncertain predictions.

Questions I would ask

1. What is the expected number of tiles to process?
2. What hardware will the service run on? CPU/GPU/RAM?
3. What metadata is available for each tile?
4. How should low-confidence predictions be handled?
5. What queries will analysts need most often?
6. How often is the model expected to be retrained or replaced?
7. What should happen when an invalid or unsupported image is uploaded?

PART 2

working code + README:

<https://github.com/sangeetaupadhye/galaxeye-tile-service>

PART 3 — Problem-Solving

1. The classifier is wrong about 30% of the time. What would I do?

I would first check the confusion matrix and class-wise performance to understand where the errors are happening. I would also inspect some wrong predictions and their confidence scores.

Then I would compare the results with the actual use case. Whether 70% accuracy is useful depends on what the predictions are being used for.

If improvement is needed, I would first check the data and labels, then preprocessing and augmentation, and finally the model.

2. How would I know the offline service is still working after a month?

I would keep local logs with timestamps, prediction counts, model version and errors.

I would also keep a few known test tiles and run them periodically to check that the service still produces the expected results.

For production, I would add a local health check as well.

3. Tiles are coming in fine, but stored results look wrong. What would I do?

I would take one incorrect result and check it step by step:

1. Run the same tile through `/classify` again.
2. Compare the API response with the SQLite row.
3. Check image preprocessing and the 64×64 resize.
4. Check the model version and loaded model.
5. Compare the probability values before and after storage.

This would tell me whether the problem is in preprocessing, inference, the API, or database storage.

4. What is the weakest part of my design?

The weakest part is the model because it is trained on a relatively small dataset and may not generalize well to different satellite imagery.

From the system side, SQLite and the single-process setup would be the first things I would revisit if the number of tiles or concurrent requests increased.

This is the version I recommend. It should fit comfortably on about **one page or less** and sounds much more like your own engineering notes.